Automated Detection of Pork Rasher Packaging Errors and Meat Quality

Alec Stanley

Student number: 33963975

MMA3001 — Numerical Methods and Machine Learning

Dataset 3 — Frozen-Meat Packaging Quality

Repository: https://github.com/alecstanley-git/mma3001-project

25 October 2026

1. The Engineering Problem

The traditional method of checking packaging and meat quality of prepackaged meat, for example pork rashers, was typically done manually. Every pack would be checked by eye. This is limited as it relies on fatigable human inspectors. It is susceptible to human error and comes with unnecessarily high labour costs. To develop a screening aid to replace this method, the system would need to effectively determine whether or not a package is defective. Ideally, this would be done from a single photograph.

In this case study, packs of pork rashers pass a fixed camera on a conveyor belt after sealing. Some packages may leave the sealer with an open seal, wrinkled film, or loose or twisted meat. This is a problem since faulty packaging could indicate spoilage and represents a food safety risk, leading to potential returns and recalls later down the line. The software, which checks each package from a single image from the camera, flags affected packaging for review. Importantly, the software does not reject packs entirely on its own. Thus, it is better to be overly cautious than the alternative.

The goal of this case study, then, is to build this automated quality inspection framework that will sit on a real production line. This is a key concept in manufacturing and automation, where mechanical engineers are challenged on ways to effectively aid in the quality inspection process by introducing intelligent automation methods. For this case study, the scope is limited to packaging defects and meat quality, where the affected points on the packaging are localised and highlighted. The only product is pre-packaged pork rashers, with images taken by a single camera and one lighting setup. This data was obtained from the publicly available dataset (Hello, 2025).

A key limitation of this data set is that it only contains 3 test examples for loose meat, and 4 for twisted meat. We would expect this to negatively affect the model's effectiveness at detecting these kinds of defects. We still include meat quality defects in the scope since we will also explore how alternative solutions can still detect these defects even with such a small training set. It is also a fact that the models trained during this case study are only applicable for pork rashers under this specific setup (camera and lighting). A new line, product, or camera would need a new training set to retrain the model.

2. Inputs, Outputs, and Data

The inputs are single colour photographs (Figure 1). These can be in the format of a `JPEG`, `PNG`, or `BMP` file, and are specifically limited to 720 by 540 pixels with each pixel storing three colour values. These colour values are stored as whole numbers, 0 to 255, and images with more or less than the required resolution are stretched to match via bilinear interpolation. The model is then required to determine localised defects. It outputs 1) the four edges of a bounding box that enclose the predicted defect, returned as fractions of the image dimensions, 2) a 0–1 confidence value per box to measure how sure the detector is of a defect, 3) the type of defect mapped by enumeration, and lastly 4) the verdict - the model needs to determine whether the entire image does or does not contain a defect. This was chosen to be the case if any box in the image contains a confidence of 0.61 or more (chosen based on the validation data). These defective packages would then be set aside for a person to check. Table 1 contains a tabulated summary of the inputs and outputs.

There are five defective classes, enumerated as 0–4 from the model outputs. Three describe the packaging (unsealed, wrinkle, packaging error), and two describe the meat (loose meat, twisted meat). The exact amounts of each class are shown in Table 9 which comes directly from the

dataset itself. The dataset contains 3,549 labelled photographs in YOLO text format, including training, validation, and test images. These are shown in Table 2. The training frames were supplied as three brightness-adjusted copies.

During the data selection, 36 training photographs were removed because another copy of the same frame is already in the validation or test part (leaving 3,513 photographs in the dataset). Any data with a malformed text label[1] or an unreadable photograph stops with an error. A missing text file is treated as 'no defect' and shows a warning. Every function's inputs and outputs are documented in the HTML documentation (`docs/`).

Table 1: The top-level input and outputs of the solution.

Item	**Format**	**Units and range**	**Meaning and use**
Input: photograph	JPEG, PNG or BMP file, 720 by 540 pixels, three colour values per pixel	whole numbers, 0 to 255; other sizes are stretched to 720 by 540	one frame from the fixed camera
Output: box	left, top, right and bottom edges	fractions of the image width and height, 0 to 1	where the defect is
Output: confidence	one number per box	0 to 1, no units	how sure the detector is
Output: defect class	one whole number per box	0 to 4: loose-meat, packaging-error, twisted-meat, unsealed, wrinkle	the kind of defect
Output: verdict	defective or clean, one per photograph	defective if any confidence is 0.61 or more	defective packs are set aside for a person to check

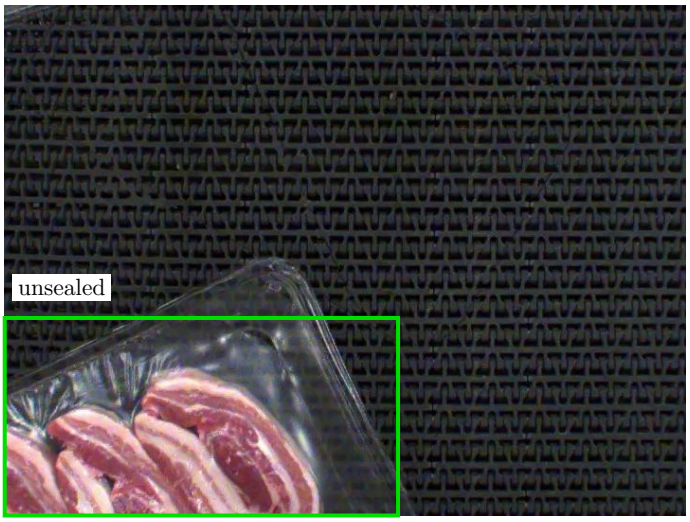


Figure 1: One test photograph with its labelled defect. Its label file holds the line `3 0.289 0.806 0.578 0.387`: the class number (3, unsealed), then the centre, width and height of the box as fractions of the image.

[1]A malformed text label would occur if there aren't five numbers in the row, the class ID is outside 0–4, or the box coordinates are outside [0, 1]

Table 2: The three parts of the dataset after removing training copies of held-out frames. Training fits the network, validation chooses settings, and test is used once for the final result.

Part	Photographs	Original frames	Labelled defects	Clean photographs
Training	3,313	1,073	2,887	684
Validation	120	120	121	14
Test	80	80	79	12

3. The Computational Approach

The chosen approach was to train the model using the YOLO format. This format is a lightweight plain-text annotation structure where each image has a corresponding text file. One image may also contain multiple defects, in which case the text file contains the bounding boxes of all the defects across multiple lines. The text file first has the class ID, the centre of the bounding box, and the normalised width and height of the bounding box (0–1). For example, `1 0.238 0.736 0.476 0.183` would represent the class ID `1` (packaging-error) with a bounding box centred on $x = 0.238$ ($x_{\text{px}} = 171.5$) and $y = 0.736$ ($y_{\text{px}} = 397.5$) with the box width as $720 \times 0.476 = 343$ px and height of $540 \times 0.183 = 99$ px.

The YOLO method treats detection as a regression problem (Redmon et al., 2016). The bounding boxes are determined by laying a 12 by 9 grid over the photograph, and then for every cell regressing a neural network to determine whether a defect exists there, its box, and its class (10 numbers per cell). The reason this method was chosen is since it can be directly built from the concepts taught in the MMA3001 class notebooks. I have shown the main method components alongside which unit notebooks cover that content in Table 3. So instead of calling a prebuilt regression library, we can demonstrate the exact topics taught and demonstrate a sound understanding of what's actually happening.

The overall structure of the network consists of a multilayer perceptron written by hand in NumPy with layers of 1,542, 128, 64, and 10 values and 206,410 learned parameters. It works by minimising the weighted squared error of Equation 1 using backpropagation and the Adam optimiser (Kingma & Ba, 2015). During training, 15% of the training frames are held back, and the training stops when the score on them stops improving (Figure 2). The network again works only under the assumptions of fixed camera and lighting, where there can only be at most one defect per 60 by 60 pixel cell. The settings are all listed in Table 10.

The hidden layers themselves use a ReLU activation function, making their outputs non-linear, while the output layer is entirely linear.

$$J = \frac{1}{2n} \sum_{i} \sum_{k} s_{ik} (y_{ik} - \hat{y}_{ik})^2 + \frac{\alpha}{2n} \|W\|_2^2 \tag{1}$$

Here y_{ik} is target k of grid cell i, $\hat{y}_{ik}$ is the prediction, W holds the weights, $\alpha = 0.05$ is the weight penalty and n is the number of cells in a batch. The weight s_{ik} switches the box and class errors off in cells that contain no defect.

The 10 outputs per cell are:

- 1 objectness score
- 4 box coordinates (cell-relative centre offsets and fractional width/height)
- 5 one-hot class scores

So for a defective cell, $s_{ik} = 1$ for objectness and class outputs and $s_{ik} = 5$ for the four box coordinates. And for an empty cell, it equals 0.5 for objectness and 0 for box and class outputs.

This method was also chosen since, in addition to it aligning closely with unit content, it is able to run on an ordinary processor without a graphics card or pretrained model. Doing it this way allows us to inspect every step, and it is fast enough that it could actually sit on a conveyor (under 6ms per photograph). It is also relatively small, again at only 206,410 parameters.

A further note on the first layer consisting of 1,542 inputs: this arises since each photograph is cut into blocks of 20 by 20 pixels (the initial 2×2 downsampling, followed by the 10×10 reduced pixel blocks), where each block is described by 14 numbers (9 edge directions, edge strength, 3 average colours, brightness spread). For each cell the network gets the 14 numbers of the 7x7 blocks around the cell (686), the same 14 numbers averaged per cell over the 7x7 cells around it (686), a coarse summary of the whole photograph, 3 by 4 regions of 14 numbers (168), and the cell's row and column (2). 686 + 686 + 168 + 2 = 1,542.

To ensure numerical stability, the inputs were scaled to average 0 and spread 1 before training, including a guard against dividing by zero spread. The initial starting weights were drawn with a spread of $\sqrt{\frac{2}{\#\text{ inputs}}}$ so the signals stay a similar size through the layers (don't blow up). Adam adds 1e-8 to avoid division by zero, and the weight penalty exists to keep weights small. Convergence is defined when 10 passes have no improvement in average precision on the held back frames, at which point training stops and the best weights are restored. The network does still have sources of numerical error whereby there is a truncation error in the central finite differences used for the edges, and 32-bit floating point rounding error in the arithmetic (the gradient check is run in 64-bit for this reason).

Table 3: Each step of the YOLO process, where it is written by hand in this project, and the unit content it uses.

Step	**Done by hand in**	**Unit content**
Read and select data	`dataset.py`	5.6 Training data evaluation and selection
Describe each image block by its edges and colour	`features.py`	Week 7 finite differences
Turn boxes into regression targets	`grid.py`	5.4 Neural network regression
Train the network	`network.py`	5.4 scaling, loss, backpropagation, Adam, early stopping
Score the detections	`metrics.py`	5.5 Evaluating performance; Week 6 trapezoid rule

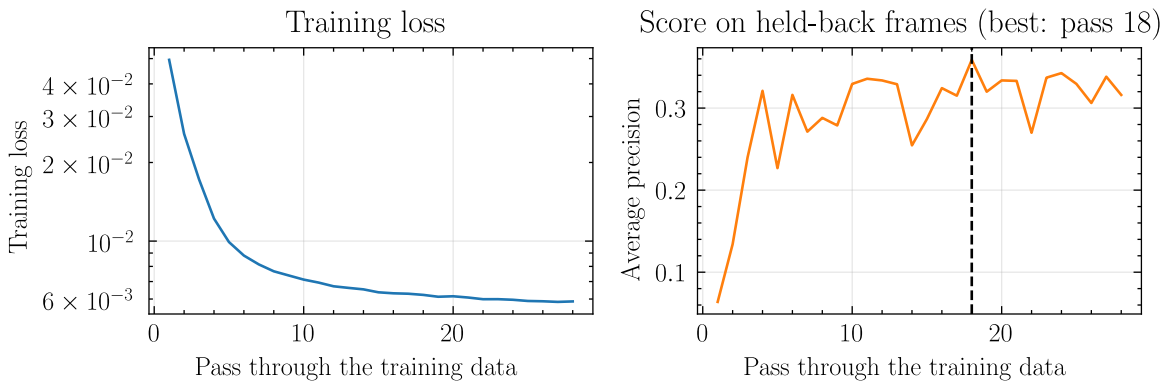


Figure 2: Training record of the hand-written detector. Left: the training loss falls steadily. Right: average precision on the held-back training frames levels off, and the weights from the best pass are kept.

4. The Validation Method and Results

This section discusses validation in the sense of proving the results of the network are trustworthy and accurate. This is not to be confused with the validation photographs in the dataset which was used to tune the network settings, including the confidence threshold. To validate the data, 80 photographs were included in the dataset for testing, scored only once after training. Withholding this test data is an effective way to validate the data since it comes with known correct answers. Future packs on the line can be fed into the network in the same way these test images were.

A reasonable criterion for the success of the model would be if it were to flag at least 70% of test cases and let at least 90% of the clean packs through. I chose this myself based on a best guess estimation of the kinds of errors human inspections might be susceptible to. A human team may let some defects through as long as most of the clean packs make it through so as not to overly affect production speed.

A detection is marked as correct when it overlaps a labelled box by at least 0.5 (shared area divided by combined area). And then the average precision is the area under the precision-recall curve (the hand-written curve in Figure 4), found using the trapezoid rule. The test average precision was found to be 0.512, which was close to the validation value of 0.529, indicating the settings were not over-fitted to the validation data. 47 of 68 defective packs are flagged and 11 of 12 clean packs pass (Figure 3). From these results, the validation study influenced the model selection whereby the 0.61 package-level confidence threshold was chosen as well as the network hyperparameters.

To verify the correctness of the code, 103 automated tests were used to check (Table 11). Backpropagation agrees with central finite differences to within $< 10^{-6}$ relative error (in 64-bit float), and predictions agree with scikit-learn's `MLPRegressor` (Pedregosa et al., 2011) given the same weights.

It was determined that the model only correctly flagged 69% of the defective packets, failing the first success criterion. It did, however, pass the second criterion through letting 92% of clean packs through.

However, the model was not tested on its performance for production on a new day. The frames, including the test frames, all still come from one continuous video and about two thirds of the held-out frames lie within five frames of a training frame.

Table 4: Results of the hand-written detector. The confidence threshold was chosen on validation and then applied unchanged to test.

Measure	Validation	Test
Average precision (overlap 0.5)	0.529	0.512
Average precision (overlap 0.75)	0.166	0.154
Precision at confidence 0.61	0.68	0.69
Recall at confidence 0.61	0.52	0.43
Defective packs flagged	78 of 106 (74%)	47 of 68 (69%)
Clean packs passed	10 of 14 (71%)	11 of 12 (92%)
Balanced accuracy of verdicts	0.73	0.80

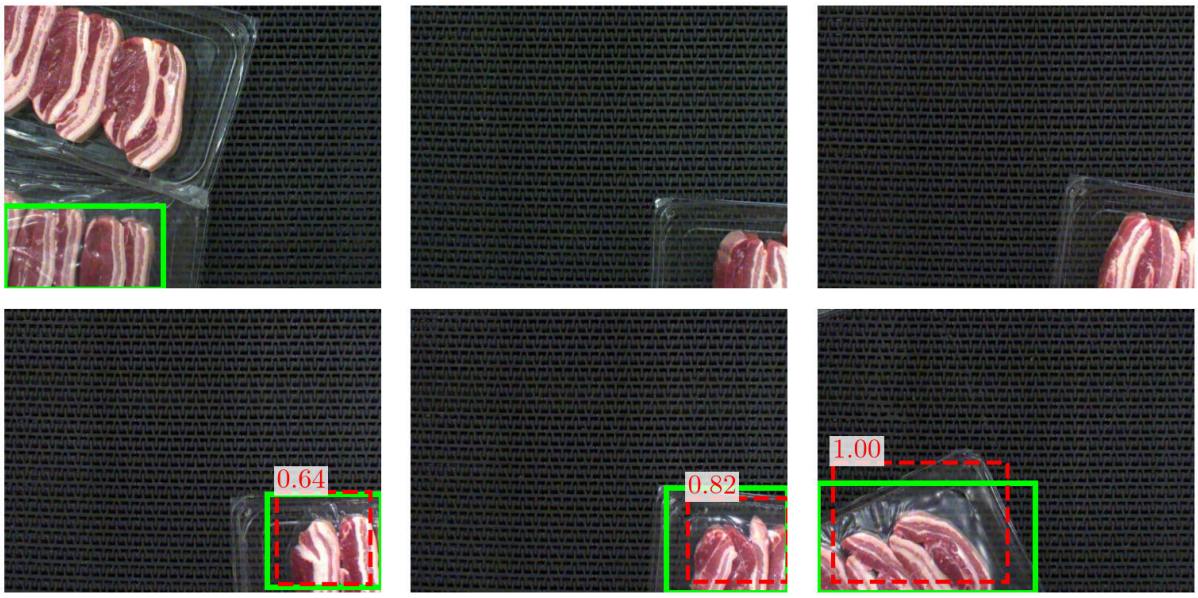


Figure 3: Six test photographs taken at even spacing, not hand-picked. Green boxes are labelled defects and red dashed boxes are detections with their confidence.

5. Comparison and Optimisation Work

5.1. Alternative solution: the Ultralytics library

A sound alternative to the hand-written NumPy network would be to use an established library specifically designed for the YOLO data format. A suitable library for this task was chosen to be the Ultralytics YOLOv8 library (Jocher et al., 2023), used as supplied in a few lines, trained on the same data and scored by the same code (Table 5, Figure 4).

We found that, overall, Ultralytics is definitively better (0.738 against 0.512). By class (Table 6) the two are close on large “unsealed” defects, but the hand-written detector misses small “packaging-error” defects, which are only about two feature blocks wide. Despite being a better detector, the Ultralytics method suffers from the cost it takes to run it. The hand-written detector has 15 times fewer parameters, trains about 20 times faster, uses about a fifth of the memory when detecting, and requires no graphics processor or pretrained model. The same cannot be said for the Ultralytics version, which trains concurrently on a graphics processor. The Ultralytics model also starts from pretrained COCO weights whereas the hand-written

weights are initially randomised. Another point of difference here is that the Ultralytics model trained on all 3,349 images in the dataset, including the 36 omitted by the hand-written model.

Table 5: The hand-written detector against the Ultralytics alternative on the 80 test photographs, scored by the same code. Ultralytics was trained on a graphics processor while other jobs were running, so its training time is approximate. Peak memory is the most held at one time while detecting all 80 photographs in one call on the processor, as reported by the operating system.

Measure (test photographs)	**Hand-written detector**	**Ultralytics YOLOv8 nano**
Average precision, any defect (overlap 0.5)	0.512	0.738
Average precision, any defect (overlap 0.75)	0.154	0.420
Mean average precision over classes	0.201	0.582
Precision at chosen confidence	0.69	0.75
Recall at chosen confidence	0.43	0.68
Defective packs flagged	47 of 68 (69%)	58 of 68 (85%)
Clean packs passed	11 of 12 (92%)	11 of 12 (92%)
Learned parameters	206,410	3,011,823
Training time (minutes)	2.4	55
Time per photograph (milliseconds)	5.9	14.7
Peak memory while detecting (gigabytes)	0.8	4.3

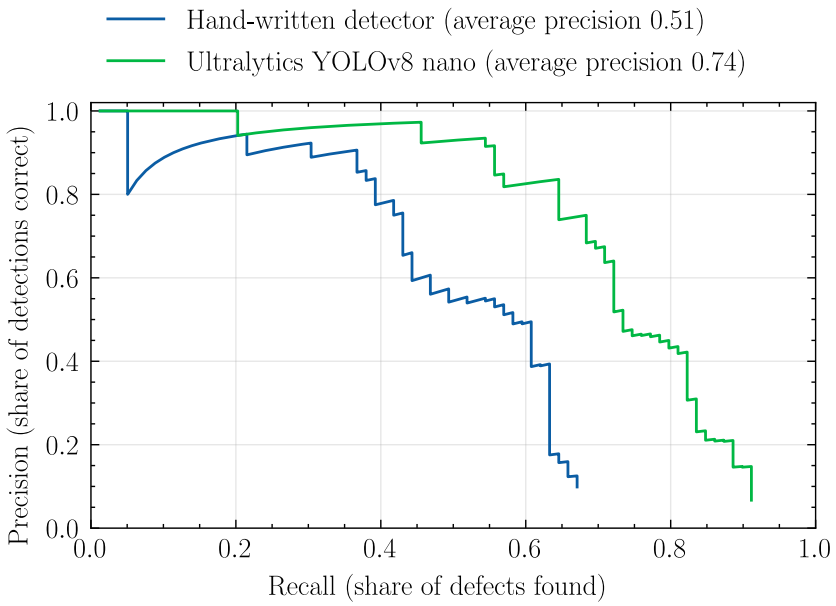


Figure 4: Precision against recall for both solutions on the test photographs.

Table 6: Average precision by defect class on the test photographs. Classes with three or four examples cannot be judged reliably.

Defect class	Test defects	Hand-written detector	Ultralytics
unsealed	49	0.781	0.838
packaging-error	23	0.021	0.485
twisted-meat	4	0.000	0.005
loose-meat	3	0.000	1.000
wrinkle	0	–	–

5.2. Effect of each design choice

We can also experiment with changing the settings of the model to see which ones affect the result the most. Referring to Table 7, it is evident that the hidden layers matter most. The other choices are each worth about 0.05, close to the spread between random seeds.

Table 7: Effect of each design choice on the validation photographs. One setting is changed at a time; the test photographs were not used for these decisions.

Variation	Average precision	Learned parameters
Chosen settings, four random seeds	0.504 to 0.549	206,410
No hidden layer (linear regression)	0.246	15,430
One hidden layer of 64	0.504	99,402
Hidden layers of 256 and 128	0.530	429,194
Weight penalty 10 times weaker	0.483	206,410
Weight penalty 10 times stronger	0.481	206,410
No mirrored copies of training photographs	0.481	206,410
Only the centre cell reports each defect	0.462	206,410

5.3. Profiling and speed-up

One of the biggest speedups was changing the way the method shrinks down images. Initially, the 2x2 averaged pixel blocks were found by quite literally diving up the entire image into 2x2 blocks, using NumPy to calculate the average colour within each block, and reconstructing a new image with those averages at half the quality. However, to achieve the same result, there was a faster way involving strided sums. Instead, for each of the four starting positions in a single 2x2 cell, you can build arrays of every second pixel in the image. This gives us four unique images: one involving pixels from the top-left of every square, one for the top-right, bottom-left, and bottom-right. Then, just add the images together and divide by four. The result is the same but this operation is much faster in NumPy. Doing this, we were able to cut the time per photograph from 9.5 to 5.9 milliseconds (Table 8), a 37.9% faster result, found by profiling with Python's profiler.

A rough estimate of the arithmetic intensity of the network can also be determined. We know the network stores numbers as 32-bit floats (4 bytes each). Detecting the defects in a single photograph runs its 108 cells through the network, which is about 412,000 operations per cell, 44.5 million in all. At the same time, about 0.8 MB of weights are read (206,410 × 4 bytes) and 0.7 MB of inputs (108 × 1,542 × 4 bytes). That comes out to about 30 operations per byte moved, so the network arithmetic is efficient and takes well under a millisecond. So, the

majority of the time per photograph is done by actually processing the image into inputs for the network, rather than running the network pass itself.

Table 8: Time to detect defects in one photograph before and after the change found by profiling.

Block averaging	**Time per photograph (milliseconds)**	**Change**
Original: reshape, then average	9.5	–
Final: strided sums	5.9	38% less time, same pixel values

5.4. Final design

It was determined that hidden layers were needed (0.246 without them), but the actual size of these hidden layers is of less importance. We saw that one layer of 64 scores 0.504 and layers of 256 and 128 score 0.530, both sitting within the seed spread of 0.504 to 0.549. The middle size of 128 and 64 was kept.

On a real packing line, Ultralytics is the better choice. It flags more defective packs (85% against 69%), passes the same share of clean packs, and still takes under 15 ms per photograph. This is more than fast enough for a production line. The hand-written detector is the better choice where training performance matters, for example if there is no graphics processor or pretrained model, or where every step must be open to inspection. That last point is extremely important for why it was chosen for this case study, as it allowed me to very clearly see my method in action and understand how the neural network worked. Ultralytics, on the other hand, is a 'black box' solution in the sense that it works extremely well but it is unclear the exact steps it is taking.

6. Limitations and Lessons Learned

A lot of the limitations encountered in this case study were inherent to the dataset itself. The frames were split at random, so the scores from both the hand-written and Ultralytics solution were optimistic. It would be more beneficial to split the data by blocks of time instead. The test set also only had 80 photographs consisting of only 12 clean packs. Rarer classes such as wrinkles, loose meat, and twisted meat could not be effectively assessed.

Another limitation encountered was due to the block-averaged features. If we set the required overlap from 0.5 to 0.75, we found that average precision fell from 0.51 to 0.15. It would be better to use finer blocks or a second finer grid, for example possibly reducing block size from 20 px to 10 px so that a defect spanning 2 blocks would now span 4.

An important lesson learned was that, through establishing a no-hidden-layer linear baseline, it was determined that the non-linear hidden layers played a very important role despite their added complexity. It was also highly useful to repeat the same run using different seeds to establish a 'seed spread' of run-to-run noise so small improvements weren't over-interpreted. As per Table 7, four random seeds were used for this.

Lastly, a very important result from the network showed that the majority of the time spent was actually on reading photographs and gathering inputs, not on the actual network arithmetic. This went against my expectations as I was anticipating the network arithmetic itself to be the most intense operation.

7. AI Use and Reflection

The primary AI tool used to aid in developing this case study was Claude Code (Anthropic), model Opus 5.5. In an approximate sense, the majority of the code was written by AI, while the report analysis was written by me.

The model was given access to specifically chosen Colab notebooks where the unit content was most relevant, as determined by myself. I tasked the tool with building a hand-written NumPy implementation based on the notebook content. The main network code in `network.py`, building the regression targets in `grid.py`, the main function in `detector.py`, and what the network sees `features.py`, were reviewed by me. The Ultralytics implementation was written by myself and then implemented as functions that `detector.py` could access by the AI agent.

All tests and HTML documentation were written by the AI agent, which also executed the start commands to run both the hand-written and Ultralytics networks and effectively organised and formatted the output data. All tables and figures in this report are built by AI using only numerical output values from the network training, validation, and testing, specifically chosen by me.

The choice to use the pork rasher dataset was my own, and the decision to build a basic NumPy YOLO process based on the Week 5 neural network regression was my own, as well as the comparison to the established Ultralytics YOLOv8 library.

The tool also helped identify key talking points on the results and listed some useful references. There is no AI-generated text in this report other than figure captions and all references were chosen by me according to my write-up in the report.

I found the AI tool to be the most effective at copying or adapting pre-existing work. This makes sense since these LLMs are trained on known information. So consequently, I found it's ability to consider the best approach ineffective. My initial prompting had the AI tool suggest the wrong dataset download format, followed by building a neural network that did not match the target scope (packaging and meat defects). I instead chose to approach this case study from the perspective of someone who already had all the building blocks (the week 5 content) and went from there. A lot of my testing was spent rerunning the network with the different parameters to verify the results, and visually inspecting images in the dataset that show what the model needs to look for.

Using AI in the development of the network greatly accelerated the learning process. I was able to avoid the intricacies of Python syntax while being able to visually see how my unit content directly applied to a real-world example. Thus, I was actually able to spend more of my time researching on this kind of implementation than I otherwise would have without AI. Another benefit I found later in the process was the ability to simply ask the LLM questions based on what it had produced. I found its explanations to be extremely sound especially when paired with the unit notebooks, where it could directly reference and explain concepts that helped me in other areas of the unit.

References

Hello. (2025, April). *Pork Rasher Error (Packaging) Dataset* [Open Source Dataset]. Roboflow Universe. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8 (Version 8.0.0) [Computer software]*. https://github.com/ultralytics/ultralytics

Kingma, D. P., & Ba, J. (2015, May). Adam: A Method for Stochastic Optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1412.6980

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

Appendix A: Defects by class

Table 9: Number of labelled defects of each class in each part of the dataset.

Defect class	Training	Validation	Test
packaging-error	1,679	34	23
unsealed	1,052	81	49
twisted-meat	72	3	4
loose-meat	66	3	3
wrinkle	18	0	0

Appendix B: Settings of the hand-written detector

Table 10: Settings used for the final hand-written detector.

Setting	Value	Meaning
Grid	12 by 9	cells of 60 by 60 pixels
Hidden layers	128, 64	neurons in each hidden layer
Learning rate	0.0003	step size of the Adam optimiser
Weight penalty α	0.05	discourages large weights
Batch size	512	grid cells per update of the weights
Box weight	5	extra weight on box errors in Equation 1
Empty cell weight	0.5	reduced weight on cells with no defect
Central fraction	0.5	part of a box whose cells report it
Mirrored copies	on	left-right and top-bottom mirror images
Held-back fraction	15%	training frames used for early stopping
Patience	10	passes without improvement before stopping
Random seed	0	makes a run repeatable

Appendix C: Automated tests

Table 11: The 103 automated tests. All pass; the report is `docs/test-report.html` in the repository.

Test file	Tests	Main things verified
`test_network.py`	18	backpropagation against finite differences; predictions against scikit-learn; known relationships are learned
`test_dataset.py`	24	label reading; malformed, missing and unreadable inputs; removal of shared frames
`test_grid.py`	12	boxes to targets and back; loss weights
`test_boxes.py`	11	overlap, removal of repeated boxes, mirroring
`test_features.py`	11	finite difference accuracy; mirrored features
`test_metrics.py`	11	trapezoid rule; average precision worked by hand
`test_detector.py`	8	whole detector on a small made-up dataset
`test_command_line.py`	5	each command runs and writes its files
`test_study.py`	2	one setting changed per variation
`test_baseline.py`	1	Ultralytics results converted correctly

Appendix D: Every run of the design-choice study

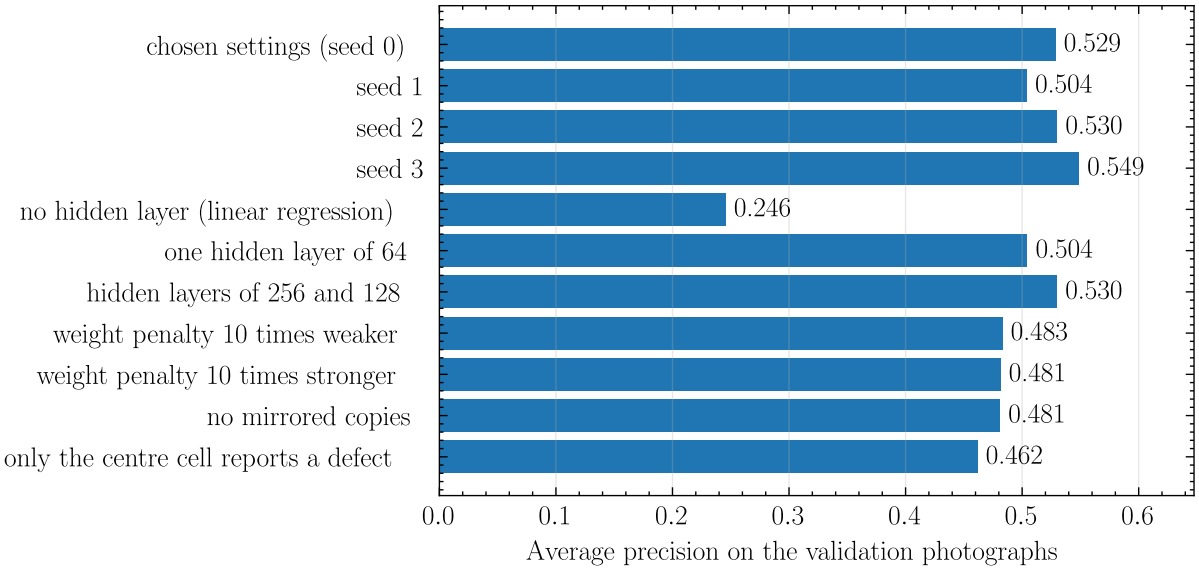


Figure 5: Validation average precision for every variation in Table 7, including each random seed.